GAME CLIENT DEVELOPER

박원국

Unity / C#

규칙의 일관성과 온라인 경험을 설계합니다.

문제를 직접 재현하고 원인을 구조로 해결한 뒤,
실제 동작으로 확인하는 개발자입니다.

2개 대표 프로젝트 · 역할과 문제 해결 중심

PORTFOLIO

CONTENTS

무엇을 만들었고, 왜 그렇게 만들었는가

01 TEAM PROJECT

Traceroute 03 - 06

4인 팀장 / 네트워크를 직접 설계하는 전략 게임
건설 비용 계산 · 패킷 만료 처리 · 업로드 완료 상태 검증

Steam 무료 데모 보기 ↗

02 SOLO PROJECT

우왕좌왕 미로: 2인·4인 협동 07 - 10

1인 개발 / 2인·4인 온라인 협동 게임
여럿이 한 캐릭터 조작 · 지연 보정 · 접속 종료 처리

Google Play에서 보기 ↗

02 핵심 역량·기술 스택

11 Contact

CORE COMPETENCIES / TECH STACK & ARCHITECTURE

사용 기술을, 문제를 해결하는 구조로 연결합니다

계산의 일관성, 입력과 응답의 순서, 자원의 수명을 구분해 클라이언트 상태를 관리했습니다.

ENGINE / LANGUAGE

Unity 6.3 / 6000.3.6f1
C# · async/await
Windows / Android

NETWORK / MULTIPLAY

Netcode for GameObjects
UGS Lobby / Relay
Client Prediction · 제한적 Rollback

ONLINE SERVICES

Cloud Save · Leaderboards
Steam 인증
Google Play Games 계정 연동

ARCHITECTURE & ENGINEERING

01 / 비용 계산 일원화

Transaction Service

설치 미리보기·확정은 공통 비용 산정 사용. 환불은 실결제 이력 기준.

사례 → p.04

02 / 진행 상태별 입력 제어

상태에 따른 입력 허용

시작 전·진행 중·종료 상태에 따라 명령 수신과 예측 이동을 제어.

사례 → p.08-09

03 / 업로드 완료 상태 확인

Version-based Validation

요청별 버전을 캡처해 이전 업로드 응답이 최신 저장의 완료 상태를 바꾸지 않게 함.

사례 → p.06

04 / 수명별 책임 분리

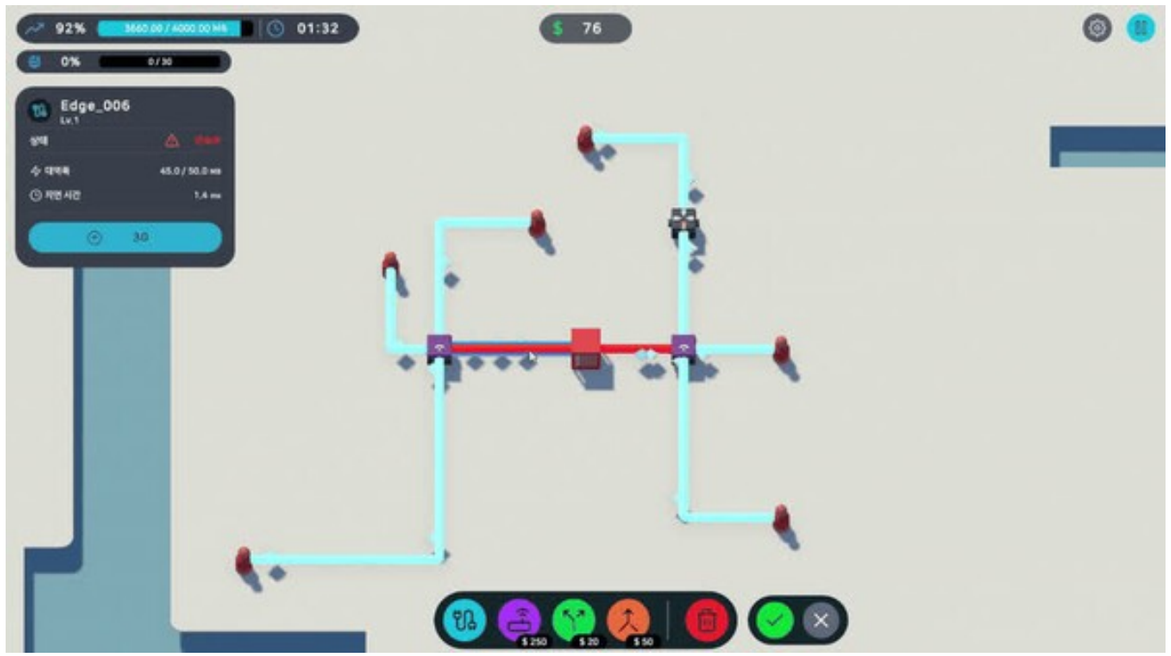
Session / Scene Lifecycle

앱 공용 데이터, 세션의 연결 자원, 씬의 UI 구독을 각 수명에 맞게 정리.

사례 → p.10

네트워크를 만드는 게임의 계산과 출시 흐름을 연결

예산 안에서 라우터와 케이블을 설치해, 패킷이 병목 없이 목적지에 도착하게 만드는 전략 게임



공식 Steam 트레일러의 실제 플레이 장면 · 패킷 이동 / 과부하 표시

내가 직접 맡은 범위

패킷 이동 경로 선택·막힘·처리 규칙

건설 비용 미리보기·결제·업그레이드·환불

플레이 스테이지·미션·튜토리얼·UI

온라인 Steam 로그인·기기/클라우드 저장·랭킹

Windows / Steam · Unity 6.3 / C#

개발 2026.02 - 2026.10 (8개월)

4인 팀 · 팀장 및 클라이언트 개발

4인
팀장 / 클라이언트 개발

0.1초
게임 규칙 계산 간격

2개 언어
한국어·영어 지원

2026.08.29
Steam 무료 데모 공개

중점 과제

건설 비용의 확정 · 만료 후 패킷의 수명
응답이 늦거나 연결이 끊겨도 진행 상황 보존

팀장으로 일한 방식

코어·건설·UI·온라인 변경 경계와 의존 관계를 정리해 작업 순서 조율
2D 지형을 건설 제한 데이터와 연결 · Git 브랜치와 PR로 변경 범위 공유

공통 비용 산정과 실결제 이력 기반 환불을 분리

설치 미리보기·확정은 같은 비용 함수를 사용하고, 철거는 실결제 이력에 따라 환불합니다.

S / 상황

미리보기·설치·철거가 금액을 각자 계산하면,
공유 셀을 중복 청구하거나 업그레이드 뒤
결제액과 환불 기준이 달라질 수 있습니다.

T / 과제

미리보기와 실제 결제의 계산 기준을 맞추고,
업그레이드·부분 철거 후에도 실제로 낸
금액을 기준으로 환불해야 했습니다.

A / 공통 비용 산정과 지불 이력 분리

① CalculateCableCost를 미리보기와 설치 확정에 함께 사용합니다.

② 이미 지불한 일반 조인트 셀은 설치 비용에서 제외합니다.

③ 업그레이드 시 등급 차액을 결제하고 셀별 지불 기록을 갱신합니다.

④ 부분 철거 시 삭제 전후에 남아 있는 지불 기록의 차액을 환불합니다.

경로 미리보기 → 공통 비용 산정

설치 확정 → 예산·연결 확인

실결제 기록 → 업그레이드 반영

부분 철거 → 지불 기록 차액 환불

R / 수정 전후의 구조 비교

수정 전 문제

단계마다 계산 결과가 달라질 수 있음
공유 셀 중복 과금 / 업그레이드 후 환불 누락 위험

구조 변경 후

미리보기와 설치는 공통 비용 산정 함수를 사용
일반 셀 중복 청구 제외 · 환불은 실결제 이력 기준

트래픽 TTL 만료와 패킷 자원 회수 시점을 분리

실패 통지는 한 번만 발행하고, 점유 자원은 각 패킷의 처리 종료에 맞춰 반환합니다.

S / 상황

요청의 제한 시간이 끝났다는 이유로 패킷까지 즉시 제거하면, 대기열과 케이블 점유가 함께 사라져 병목이 저절로 해소될 수 있습니다.

T / 과제

요청의 성공·실패 확정과 패킷의 자원 반환을 분리하고, 전체 패킷 처리가 끝난 뒤에만 요청 객체를 회수해야 했습니다.

A / 요청과 패킷의 종료 조건 분리

PostTick에서 시뮬레이션 시간이 expiresAt 이상이면 Expired로 전환합니다. outcomeReported로 실패 결과의 중복 발행을 막습니다.



R / 실패 결과와 자원 반환의 책임 분리

만료 시 실패 결과만 발행하며, 이후 각 패킷은 도착·드롭 처리에 따라 개별 반환됩니다. 전체 처리가 끝나면 요청을 회수합니다. 반환된 패킷의 재통지는 활성 검사로 제외합니다.

업로드 요청 버전으로 완료 상태 갱신을 제한

로컬에 먼저 저장하고, 요청별 JSON과 버전을 캡처해 지연 응답의 반영 범위를 제한합니다.

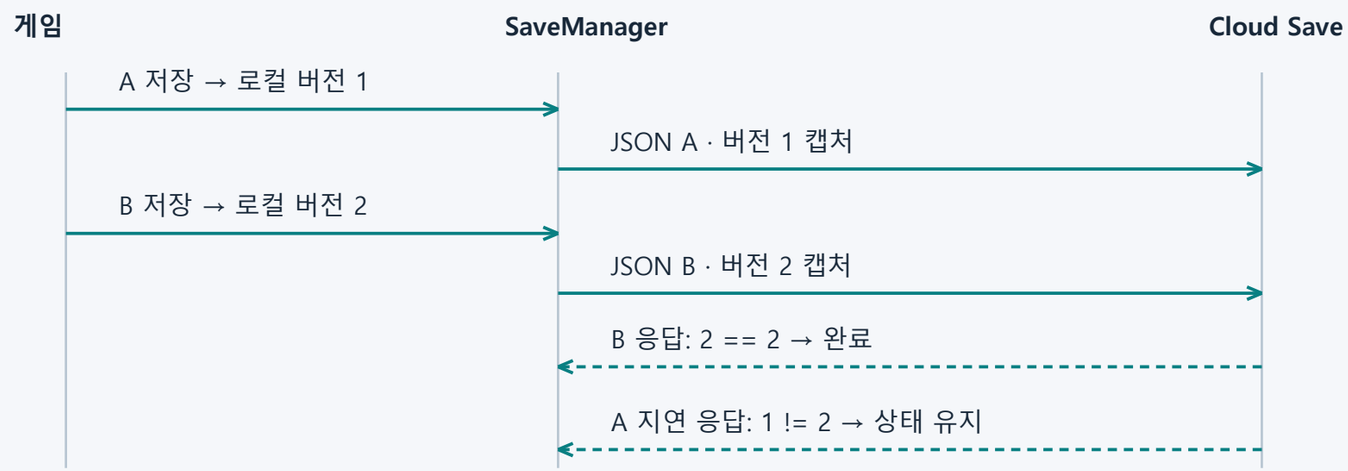
S / 상황

저장 A를 업로드하는 동안 B를 새로 저장할 수 있습니다. A의 응답이 나중에 오면, B까지 업로드된 것처럼 로컬 완료 상태를 바꿀 위험이 있습니다.

T / 과제

연결 실패에도 진행 상황은 기기에 남기고, 최신 저장과 같은 버전의 응답만 업로드 완료 상태를 갱신해야 했습니다.

A / 요청별 스냅샷과 응답 버전 비교



SaveManager.cs / SaveCloudJsonAsync 실제 코드 발췌

```
await cloudStorage.SaveAsync(SaveKey, json);
if (version == localSaveVersion)
{
    Data.hasPendingCloudSave = false;
    SaveLocalWithoutVersionChange();
}
```

R / 이전 응답으로 최신 로컬 완료 상태를 덮어쓰지 않도록 처리

Save / SaveAsync의 성공·실패 응답에 버전 비교를 적용했습니다. 보호 대상은 로컬 완료 상태이며, 클라우드 쓰기 순서나 서버 측 충돌 해결을 보장하는 기능과는 구분합니다.

한 캐릭터를 여러 명이 조작할 때의 일관성

무작위로 나눈 W·A·S·D 입력을 맞춰 별을 수집하고 목표 지점에 도달하는 온라인 협동 게임



실제 2인 플레이 화면 · 공유 캐릭터와 참가자별 담당 키 표시

1인 개발 / 직접 담당

게임 기획 · 클라이언트·온라인 기능

UI 구현 · 개인 기록 저장 · Android 빌드·배포

Unity 6000.3.6f1 / C#

Netcode for GameObjects

UGS Lobby / Relay

개발 2025.12 - 2026.10 (10개월)

Google Play 정식 출시

2인 / 4인

방향키를 나눠 맡음

1개 캐릭터

여러 참가자의 입력 통합

1인 개발

기획·구현·빌드·배포

3단계 정리

앱 / 세션 / 씬

핵심 과제

로컬 입력 반응과 호스트의 최종 판정을 분리하고, 확정 상태를 기준으로 클라이언트의 이동을 다시 계산했습니다.

내 입력은 먼저 반영하고, 최종 상태는 호스트가 확정

Client Prediction과 확정 상태 기반 재계산으로, 로컬 반응과 최종 판정의 책임을 나눴습니다.

S / 상황

호스트 응답을 기다리면 조작이 늦어지고, 예측 결과만 유지하면 다른 참가자의 입력이 반영된 최종 상태와 어긋날 수 있습니다.

T / 과제

오래된 상태와 이미 처리된 명령을 구분해 제거하고, 최신 확정 상태에서 미처리 자기 입력만 다시 적용해야 했습니다.

A / 식별값의 역할과 처리 순서

Revision: 확정 상태의 순서. 더 최신인 상태만 수용합니다.
Sequence / Ack: 입력 순서와 호스트 처리 완료 번호. 거부된 명령도 제거합니다.
Epoch: 플레이 회차. 호스트는 이전 회차 입력을 거부하고,
클라이언트는 회차 변경 시 미처리 입력과 정지 플래그를 초기화합니다.



① Revision 검사 → 확정 상태 저장 → Epoch 확인
② 같은 회차: Ack 이하 명령 제거 / 새 회차: 대기 명령 비우기
③ 위치·방향을 확정 상태 T로 복원 → 미처리 입력만 재적용

R / 예측의 누적 오차와 처리된 자기 입력의 재적용 방지

매번 확정 상태에서 다시 계산하며, 상대 입력은 호스트 상태를 통해 반영합니다. 논리 위치 갱신과 화면 표시 보정을 분리했습니다. 명령 번호로 입력·로컬 적용·호스트 처리·화면 반영 로그를 연결했습니다.

지연 입력은 제한된 이력 안에서만 다시 계산합니다

이력·판정 경계로 적용 시점을 제한하고, 과거에 새 게임 판정이 생기면 현재 틱 적용으로 전환합니다.

S / 상황

늦게 도착한 방향 전환으로 이동 경로가 달라집니다. 과거 경로를 바꾸면서 별 수집이나 도착 판정까지 새로 발행하면 게임 결과의 일관성이 깨집니다.

T / 과제

일반 이동 구간의 입력은 보정하되, 이미 확정된 판정과 외부 이벤트를 유지해야 했습니다.

설계 정책 / 이력 상한 0.2초

과거 경로 변경과 재계산 범위를 제한하는 정책값입니다. 측정으로 도출한 최적값은 아닙니다.

A / 상태 복원과 부작용 발행 경계

요청 틱은 보관 이력~현재 틱으로 제한해 모든 참가자 입력을 재생합니다. 확정 판정 이전 이력은 폐기합니다.

① **재계산 대상**: 실제 State(위치·방향)와 접촉 ID 집합을 이력으로 바꿉니다. 별도 임시 State는 두지 않습니다.

② **발행 경계**: 수집·도착·위험 이벤트는 재계산 중 발행하지 않습니다. 외부 수집·이벤트 상태는 변경하지 않아 복원 대상이 아닙니다.

③ **취소 후**: 과거 틱의 새 접촉 시 현재 위치·방향·접촉 ID를 복원하고, 이번 수신 입력 전체를 현재 틱에 적용합니다. 현재 틱 결과만 반환합니다.

HostMoveAuthority.Step / 실제 코드 발췌·중간 생략

```
if (!ReplayFrames(firstChanged, before, overlaps, out var outcome))
{
    // ... omitted: currentFrame setup and history cleanup
    currentFrame.Commands.AddRange(incoming);
    ReplayFrames(0, presentBefore, presentOverlaps, out outcome);
}
```

R / 단일 실행의 관측값 · 재계산 시간 범위

81건 / 4~11틱 → 최대 약 183ms = 11 ÷ 60초 (되감은 시뮬레이션 범위, RTT·연산 시간 아님)

2026.10.25 · Android 앱 1.8 · SM-A366N · 2인 Map_02 · 호스트 로그 225.392초 · 접촉 취소 관측 0건

수명별 책임과 실제 종료 순서를 구분합니다

이탈 정책을 정하고, 중복 통지와 이전 세션의 지연 응답이 새 세션을 건드리지 않게 처리했습니다.

S / 상황

대기실의 참가자 이탈은 방을 유지할 수 있지만, 게임 중 이탈은 공유 조작을 중단해야 합니다. 종료 통지와 비동기 응답도 뒤늦게 도착할 수 있습니다.

T / 과제

자원의 소유 수명과 종료 순서를 분리하고, 이미 끝난 세션의 정리나 화면 전환이 반복되지 않는지 확인해야 했습니다.

이탈 주체별 정책

게임 중 호스트·참가자 이탈
남은 플레이어도 세션 종료·로비 복귀
대기실 참가자 이탈: 호스트는 방 유지
대기실 호스트 이탈: 참가자는 종료·복귀

종료 후 Current가 null이면 정리 생략.
Scope.Dispose는 isDisposed로 재진입 차단.

A / 책임 구조 (소유 수명)

앱 / AppScope
프로필·공용 Store

세션 / SessionScope
방·중계·네트워크

씬 / EntryPoint
UI·입력 구독

A / 실제 종료 시퀀스 (LeaveSessionUseCase)

방 이탈 요청 시작(응답 대기 안 함) → Runner.Shutdown → Store.Clear
→ SessionLifetime.End(Current 해제·Scope 정리) → 종료 이유 보관 → 로비 로드

이전 응답 거부: 게임 시작 요청은 await 전후에 세션 수명 취소 여부와 ReferenceEquals(요청 Scope, 현재 Scope)를 확인합니다.

R / 대표 검사 3개 · 조건 → 기대 결과 → 확인 결과

방 생성 후 씬 전환 거절
기대: 호스트·세션 정리 후 재시도 가능 — **확인: 종료 1회, 재시도 성공**

게임 시작 응답 대기 중 세션 교체
기대: 이전 요청의 씬 전환 차단·새 세션 유지 — **확인: 로드 0회, 새 Scope 유지**

종료 대상 3분기에서 이탈 통지 2회
기대: 중복 정리와 중복 로비 이동 방지 — **확인: 종료·로비 로드 각 1회**

프로필·저장 순서·세션 등 17개 시나리오 통과

CheckV2.csproj가 제품 Domain/Application 소스를 직접 컴파일. 서비스·씬 로더는 대역 사용.
Unity Play Mode·UGS 실통신·다중 기기 결과 일치 검증과 구분합니다.

NEXT CHALLENGE

함께 만들 다음 플레이를 기대합니다.

"시뮬레이션 규칙과 비용 계산의 일관성부터,
지연 입력과 비동기 응답의 순서까지
상태가 어긋나는 문제를 구조로 풀어 왔습니다.

문제를 재현하고 원인을 확인하며,
플레이어가 믿고 즐길 수 있는
게임을 만드는 클라이언트 개발자가 되겠습니다."

박원국

Game Client Developer · Unity / C#

CONTACT & PROJECT LINKS

01 GitHub 프로필 ↗
github.com/parkwonkuku

02 Traceroute
Steam 무료 데모 ↗ · GitHub 코드 ↗

03 우왕좌왕 미로
Google Play ↗ · GitHub 코드 ↗

EMAIL
dnjsrnrdl456@gmail.com

PHONE
010-6308-3703